

主要内容

- 基本语法
- 数据类型
- 运算符
- 标准库

强制类型转换

- 使用**String()**将任意类型的值强制转换成字符串
 - 参数是原始类型值，转换规则如下：
 - 数值：转为相应的字符串
 - 字符串：转换后还是原来的值
 - 布尔值：**true**转为字符串"**true**"，**false**转为字符串"**false**"
 - **undefined**：转为字符串"**undefined**"
 - **null**：转为字符串"**null**"

```
String(123) // "123"  
String('abc') // "abc"  
String(true) // "true"  
String(undefined) // "undefined"  
String(null) // "null"
```

强制类型转换

- 使用String()将任意类型的值强制转换成字符串

- 参数是对象，转换规则如下：

- 参数如果是狭义对象，返回一个类型字符串
- 参数如果是函数，返回函数.toString值
- 参数如果是数组，返回该数组的字符串形式

```
>> String({a: 1})  
← "[object Object]"  
  
>> function f(){ return null;}  
← undefined  
  
>> String(f)  
← "function f(){ return null;}"  
  
>> f.toString()  
← "function f(){ return null;}"  
  
>> String([1,2,3])  
← "1,2,3"
```

强制类型转换

- 使用 **Boolean()** 将输入的各种类型的值强制转换成布尔值
 - 转换规则相对简单：除了以下五个值的转换结果为 **false**，其他的值全部为 **true**
 - undefined
 - null
 - -0 或 +0
 - NaN
 - ""（空字符串）

```
Boolean(undefined) // false
Boolean(null) // false
Boolean(0) // false
Boolean(NaN) // false
Boolean('') // false
```

自动类型转换

- 自动转换是以强制转换为基础的，遇到以下三种情况时，JavaScript 会自动转换数据类型，即转换是自动完成的，用户不可见
 - 第一种情况，不同类型的数据互相运算
 - 第二种情况，对非布尔值类型的数据求布尔值
 - 第三种情况，对非数值类型的值使用一元运算符（即+和-）
- 自动转换的规则是：预期什么类型的值，就调用该类型的转换函数

自动类型转换

- 自动转换为布尔值

- JavaScript 遇到预期为布尔值的地方（比如if语句的条件部分），就会将非布尔值的参数自动转换为布尔值。系统内部会自动调用Boolean函数

```
if ( !undefined
    && !null
    && !0
    && !NaN
    && !''
) {
    console.log('true');
} // true
```

自动类型转换

- 自动转换为字符串

- JavaScript 遇到预期为字符串的地方，就会将非字符串的值自动转为字符串，主要发生在字符串的加法运算时
- 具体规则是，先将复合类型的值转为原始类型的值，再将原始类型的值转为字符串

```
'5' + 1 // '51'  
'5' + true // "5true"  
'5' + false // "5false"  
'5' + {} // "5[object Object]"  
'5' + [] // "5"  
'5' + function (){} // "5function (){}"  
'5' + undefined // "5undefined"  
'5' + null // "5null"
```

自动类型转换

- 自动转换为数值

- JavaScript 遇到预期为数值的地方，会将参数值自动转换为数值，系统内部会自动调用Number函数
- 除了加法运算符 (+) 有可能把运算符转为字符串，其他运算符都会把运算符自动转成数值

```
'5' - '2' // 3
'5' * '2' // 10
true - 1 // 0
false - 1 // -1
'1' - 1 // 0
'5' * [] // 0
false / '5' // 0
'abc' - 1 // NaN
null + 1 // 1
undefined + 1 // NaN
```

```
+'abc' // NaN
-'abc' // NaN
+true // 1
-true // 0
```

```
> null + '1'
< 'null1'
> null + 1
< 1
```

null转为数值时为0，而undefined
转为数值时为NaN

主要内容

- 基本语法
- 标准库

URI编解码函数

- ECMAScript 语言自身提供四个函数用来编码和解码 URI
 - `encodeURIComponent`
 - `decodeURIComponent`
- `encodeURIComponent`会将 URI 中那些不合法的字符（如空格、特殊符号等）进行转义处理，而保留合法的字符不变。

```
const uri = "https://www.example.com/search?query=你好 世界";  
const encodedUri = encodeURIComponent(uri);  
console.log(encodedUri);
```

`https://www.example.com/search?query=%E4%BD%A0%E5%A5%BD%20%E4%B8%96%E7%95%8C`

URI编解码函数

- 不合法或需要编码的字符：
 - 控制字符：ASCII 码值在 0 到 31 之间的字符（如回车、换行符）。
 - 空格：空格字符在 URI 中是不合法的，需要编码为 %20。
 - 非 ASCII 字符：如中文字符、日文字符等，需要进行 URL 编码（转为 UTF-8 编码的字符并使用百分号编码）。
 - 以下特殊字符（部分符号需要视情况而定）：
 - %（百分号）&（和号）?（问号）/（斜杠）:（冒号）=（等号）
 - +（加号）;（分号），（逗号）@（小于符号）[]（方括号）#（哈希符号）
 - {}（大括号）|（竖线）"（双引号）<（小于符号）>（大于符号）

URI编解码函数

- 以下特殊字符（部分符号需要视情况而定）：

```
const uri = "https://www.example.com/search?query=hello?world";  
const encodedUri = encodeURIComponent(uri);  
console.log(encodedUri);  
// 输出: https://www.example.com/search?query=heLlo%3FworLd
```

```
const uri = "https://www.example.com/search?user=hello@world";  
const encodedUri = encodeURIComponent(uri);  
console.log(encodedUri);  
// 输出: https://www.example.com/search?user=heLlo%40worLd
```

URI编解码函数

- encodeURI 函数操作的是完整的 URI，encodeURIComponent 函数操作的是组成 URI 的组件。如果链接内包含一个回调地址，回调地址是另外一个 URL，此时我们就需要使用 encodeURIComponent() 对回调地址进行转码，这样一来，URL 中就不会出现多个 http://，多个 & 这样的特殊字符；方便对回调地址进行处理；

```
1 console.log(encodeURI("http://example.com/pagephp?id=123&age=23&sex=\\gril"));
2 // http://example.com/pagephp?id=123&age=23&sex=%5Cgril
```

```
1 https://www.baidu.com/s?returnURL=http://www.test.com/
2
```

```
1 // 对URL中的回调链接进行转码
2 'https://www.baidu.com/s?returnURL=' +encodeURIComponent('http://www.test.com/')
3 //输出: "https://www.baidu.com/s?returnURL=http%3A%2F%2Fwww.test.com%2F"
4
```

字符串对象String

• match方法

- 用于确定原字符串是否匹配某个子字符串。匹配返回一个数组，它包含了第一个匹配的详细信息。没有匹配返回空数组

```
> var matches = 'cat, bat, sat, fat'.match('at');  
< undefined  
> matches  
< ► ['at', index: 1, input: 'cat, bat, sat, fat', groups: undefined]  
> matches[0]  
< 'at'  
> matches[1]  
< undefined
```

g (global) : 表示全局匹配，查找字符串中所有符合条件的结果。

i (ignore case) : 表示忽略大小写匹配。

```
> var str = "cat, bat, sat, fat."  
var pattern = /at/gi;  
  
var matches = str.match(pattern);  
console.log(matches);  
  
► (4) ['at', 'at', 'at', 'at']
```

字符串对象String

- search方法

- 用法基本等同于match，但是返回值为匹配的第一个位置，如果没有找到匹配，则返回-1

- replace方法

- 用于替换匹配的子字符串，一般情况下只替换第一个匹配（除非使用带有g修饰符的正则表达式）

```
>> 'cat, bat, sat, fat'.search('ba')  
< 5  
>> 'cat, bat, sat, fat'.search('tt')  
< -1  
>> 'cat, bat, sat, fat'.replace('at', 'tt')  
< "ctt, bat, sat, fat"
```

```
> 'cat, bat, sat, fat.'.replace(/at/, 'xx')  
< 'cxx, bat, sat, fat.'  
> 'cat, bat, sat, fat.'.replace(/at/g, 'xx')  
< 'cxx, bxx, sxx, fxx.'
```

字符串对象String

- charAt方法

- 返回指定位置的字符，参数是从0开始编号的位置
- 如果参数为负数，或大于等于字符串的长度，返回空字符串

```
>> s='abc'  
← "abc"  
>> s.charAt(s.length-1)  
← "c"  
>> s.charAt(-1)  
← ""  
>> s.charAt(100)  
← ""
```


字符串对象String

- **concat方法**

- 用于连接两个字符串，返回一个新字符串，不改变原字符串
- 如果参数不是字符串，concat方法会将其先转为字符串，然后再连接

```
>> var s1 = 'abc';  
    var s2 = 'def';  
    s1.concat(s2)  
← "abcdef"  
  
>> s1  
← "abc"  
  
>> s1.concat('tt','xx')  
← "abcttxx"  
  
>> s1.concat(22,33)  
← "abc2233"
```

字符串对象String

- slice方法

- 用于从原字符串取出子字符串并返回，不改变原字符串，第一个参数是子字符串的开始位置，第二个参数是子字符串的结束位置（不含该位置）

```
>> 'JavaScript'.slice(0, 4)
< "Java"
>> 'JavaScript'.slice(4)
< "Script"
>> 'JavaScript'.slice(3, 1)
< ""
>> 'JavaScript'.slice(-6)
< "Script"
>> 'JavaScript'.slice(0, -6)
< "Java"
>> 'JavaScript'.slice(-3, -2)
< "i"
```

- 如果省略第二个参数，表示子字符串一直到字符串结束
- 如果参数是负值，表示从结尾开始倒数计算的位置，即该负值加上字符串长度
- 如果第一个参数大于第二个参数，返回一个空字符串

字符串对象String

- indexOf方法

- 用于确定一个字符串在另一个字符串中第一次出现的位置，返回结果是匹配开始的位置。如果返回-1，就表示不匹配

```
>> 'hello world'.indexOf('o')  
← 4  
>> 'hello world'.indexOf('World')  
← -1  
>> 'hello world'.indexOf('world')  
← 6
```

区分大小写

- lastIndexOf从尾部开始匹配，indexOf则是从头部开始匹配

```
> 'hello world' .lastIndexOf('o')  
← 7
```

字符串对象String

- **trim**方法

- 用于去除字符串两端的空格，返回一个新字符串，不改变原字符串
- 该方法去除的不仅是空格，还包括制表符（\t、\v）、换行符（\n）和回车符（\r）

```
>> '\r\n    abc    \t'.trim()  
← "abc"
```

字符串对象String

- **toLowerCase**方法用于将一个字符串全部转为小写，**toUpperCase**则是全部转为大写，它们都返回一个新字符串，不改变原字符串

```
>> 'Hello World'.toLowerCase()  
← "hello world"  
  
>> 'Hello World'.toUpperCase()  
← "HELLO WORLD"
```

字符串对象String

- **split**方法按照给定规则分割字符串，返回一个由分割出来的子字符串组成的数组

```
>> 'a|b|c'.split('|')
< ▶ Array(3) [ "a", "b", "c" ]
>> 'a|b|c'.split('')
< ▶ Array(5) [ "a", "|", "b", "|", "c" ]
>> 'a|b|c'.split()
< ▶ Array [ "a|b|c" ]
>> '|a||c|'.split("|")
< ▶ Array(5) [ "", "a", "", "c", "" ]
>> 'a|b|c'.split('|',2)
< ▶ Array [ "a", "b" ]
```

- 如果分割规则为空字符串，则返回数组的成员是原字符串的每一个字符
- 如果省略参数，则返回数组的唯一成员就是原字符串
- 如果满足分割规则的两个部分紧邻着或位于开头结尾，则返回数组之中会有一个空字符串
- 可以接受第二个参数，限定返回数组的最大成员数

字符串对象String

- localeCompare方法

- 用于比较两个字符串，返回一个整数，如果小于0，表示第一个字符串小于第二个字符串；如果等于0，表示两者相等；如果大于0，表示第一个字符串大于第二个字符串
- 该方法的最大特点，就是会**考虑自然语言的顺序**

```
>> 'a' > 'B'  
← true  
  
>> 'a'.localeCompare('B')  
← -1
```

因为 JavaScript 采用的是 Unicode 码点比较，B的码点是66，而a的码点是97，字母B小于字母a

localeCompare方法返回整数-1，表示B较大，考虑了自然语言的顺序

日期对象Date

- **Date**对象是 JavaScript 原生的时间库，它以1970年1月1日00:00:00作为时间的零点，可以表示的时间范围是前后各1亿天（单位为毫秒）
- **Date**对象可以作为普通函数直接调用，返回一个代表当前时间的字符串

```
> Date()
```

```
< 'Tue Dec 12 2023 20:58:44 GMT+0800 (中国标准时间)'
```


日期对象Date

- **Date.now**方法

- 返回当前时间距离时间零点（1970年1月1日 00:00:00 UTC）的毫秒数

- **Date.parse**方法

- 用来解析日期字符串，返回该时间距离时间零点（1970年1月1日 00:00:00）的毫秒数

- **Date.UTC**方法

- 接受年、月、日等变量作为参数，返回该时间距离时间零点（1970年1月1日 00:00:00 UTC）的毫秒数

```
// 格式  
Date.UTC(year, month[, date[, hrs[, min[, sec[, ms]]]])
```

日期对象Date

```
>> Date.now()
← 1544827068412

>> Date.parse('January 26, 2018 13:51:50')
← 1516945910000

>> Date.parse('Januaryxxxxx')
← NaN

>> Date.UTC(2011, 0, 1, 2, 3, 4, 567)
← 1293847384567    2011-01-01 02:03:04.567 (UTC)
```

日期对象Date

- Date还可以当作构造函数使用。对它使用new命令，会返回一个Date对象的实例
 - 如果不加参数，实例代表的就是当前时间
 - 可以接受多种格式的参数，返回一个该参数对应的时间实例

```
> Date()
< 'Mon Dec 11 2023 19:52:34 GMT+0800 (中国标准时间)'
```

```
> Date()-1
< NaN
```

Date()返回的是字符串

```
> new Date()-1
< 1702295585969
```

new Date()返回的是时间对象

```
> new Date()
< Mon Dec 11 2023 19:59:11 GMT+0800 (中国标准时间)
```

```
// 参数为时间零点开始计算的毫秒数
new Date(1378218728000)
// Tue Sep 03 2013 22:32:08 GMT+0800 (CST)

// 参数为日期字符串
new Date('January 6, 2013');
// Sun Jan 06 2013 00:00:00 GMT+0800 (CST)

// 参数为多个整数，
// 代表年、月、日、小时、分钟、秒、毫秒
new Date(2013, 0, 1, 0, 0, 0, 0)
// Tue Jan 01 2013 00:00:00 GMT+0800 (CST)
```

日期对象Date

- 日期的运算

- 类型自动转换时，Date实例如果转为数值，则等于对应的毫秒数；如果转为字符串，则等于对应的日期字符串
- 两个日期实例对象进行减法运算时，返回的是它们间隔的毫秒数；进行加法运算时，返回的是两个字符串连接而成的新字符串

```
var d1 = new Date(2000, 2, 1);
```

```
var d2 = new Date(2000, 3, 1);
```

```
d2 - d1
```

```
// 2678400000
```

```
d2 + d1
```

```
// "Sat Apr 01 2000 00:00:00 GMT+0800 (CST)Wed Mar 01 2000 00:00:00 GMT+0800 (CST)"
```

日期对象Date

- **Date**的实例对象常见的方法可以分为以下三类
 - **to类**：从**Date**对象返回一个表示指定时间的字符串

方法	说明
<code>toString()</code>	将日期时间转换为普通字符串
<code>toUTCString()</code>	将日期时间转换为世界时间 (UTC) 格式的字符串
<code>toLocaleString()</code>	将日期时间转换为本地时间格式的字符串

```
> new Date().toString()
< 'Tue Dec 12 2023 21:03:45 GMT+0800 (中国标准时间)'
```

```
> new Date().toUTCString()
< 'Tue, 12 Dec 2023 13:03:56 GMT'
```

```
> new Date().toLocaleString()
< '2023/12/12 21:04:13'
```

```
> Date().toString()
< 'Tue Dec 12 2023 21:04:28 GMT+0800 (中国标准时间)'
```

```
> Date().toUTCString()
✖ ▶Uncaught TypeError: Date(...).toUTCString is not a function
   at <anonymous>:1:8
```

```
> Date().toLocaleString()
< 'Tue Dec 12 2023 21:05:06 GMT+0800 (中国标准时间)'
```

由于 `Date()` 返回的是字符串，它不具备 `Date` 对象的方法，比如 `toUTCString()`

日期对象Date

- Date的实例对象常见的方法可以分为以下三类
 - get类：获取Date对象的日期和时间。

方法	说明	
getFullYear()	返回一个表示年份的4位数字	getDate
getMonth()	返回值是0（一月）到11（十二月）之间的一个整数	getDay
getDate()	返回值是1~31之间的一个整数	getFullYear
getHours()	返回值是0~23之间的一个整数，来表示小时数	getHours
getMinutes()	返回值是0~59之间的一个整数，来表示分钟数	getMilliseconds
getSeconds()	返回值是0~59之间的一个整数，来表示秒数	getMinutes
		getMonth
		getSeconds
		getTime
		getTimezoneOffset
		getUTCDate
		getUTCDay
		getUTCFullYear
		getUTCHours
		getUTCMilliseconds
		getUTCMinutes
		getUTCMonth
		new Date().get

日期对象Date

- Date的实例对象常见的方法可以分为以下三类
 - set类：设置Date对象的日期和时间

方法	说明
setFullYear()	可以设置年、月、日
setMonth()	可以设置月、日
setDate()	可以设置日数
setHours()	可以设置时、分、秒、毫秒
setMinutes()	可以设置分、秒、毫秒
setSeconds()	可以设置秒、毫秒

tab

setDate
setFullYear
setHours
setMilliseconds
setMinutes
setMonth
setSeconds
setTime
setUTCDate
setUTCFullYear
setUTCHours
setUTCMilliseconds
setUTCMinutes
setUTCMonth
setUTCSeconds
setYear

日期对象Date

- to 类方法

```
>> var d = new Date(2018, 0, 1); // 月份从0开始, 2018年1月1日
< undefined
>> d.toString() //toString方法返回一个完整的日期字符串
< "Mon Jan 01 2018 00:00:00 GMT+0800 (中国标准时间)"
>> d.toUTCString() //toUTCString方法返回对应的 UTC 时间, 也就是比北京时间晚8个小时
< "Sun, 31 Dec 2017 16:00:00 GMT"
>> d.toDateString() //toDateString方法返回日期字符串 (不含小时、分和秒)
< "Mon Jan 01 2018"
>> d.toTimeString() //toTimeString方法返回时间字符串 (不含年月日)
< "00:00:00 GMT+0800 (中国标准时间)"
>> d.toLocaleString('en-US') //将 Date 实例转为表示本地时间的字符串
< "1/1/2018, 12:00:00 AM"
>> d.toLocaleString('zh-CN')
< "2018/1/1 上午12:00:00"
```


日期对象Date

- get 类方法

- getTime(): 返回实例距离1970年1月1日00:00:00的毫秒数，等同于valueOf方法
- getDate(): 返回实例对象对应每个月的几号（从1开始）
- getDay(): 返回星期几，星期日为0，星期一为1，以此类推
- getFullYear(): 返回四位的年份
- getMonth(): 返回月份（0表示1月，11表示12月）
- getHours(): 返回小时（0-23）
- getMilliseconds(): 返回毫秒（0-999）
- getMinutes(): 返回分钟（0-59）
- getSeconds(): 返回秒（0-59）
- getTimezoneOffset(): 返回当前时间与 UTC 的时区差异，以分钟表示，返回结果考虑到了夏令时因素

日期对象Date

- get类方法

```
>> function leftDays() {  
    var today = new Date();  
    var endYear = new Date(today.getFullYear(), 11, 31, 23, 59, 59, 999);  
    var msPerDay = 24 * 60 * 60 * 1000;  
    return Math.round((endYear.getTime() - today.getTime()) / msPerDay);  
}  
  
← undefined  
  
>> leftDays()  
← 17
```

日期对象Date

- **set 类方法**

- **setDate(date)**: 设置实例对象对应的每个月的几号（1-31），返回改变后毫秒时间戳
- **setFullYear(year [, month, date])**: 设置四位年份
- **setHours(hour [, min, sec, ms])**: 设置小时（0-23）
- **setMilliseconds()**: 设置毫秒（0-999）
- **setMinutes(min [, sec, ms])**: 设置分钟（0-59）
- **setMonth(month [, date])**: 设置月份（0-11）
- **setSeconds(sec [, ms])**: 设置秒（0-59）
- **setTime(milliseconds)**: 设置毫秒时间戳

日期对象Date

- set 类方法

```
>> var d = new Date ('January 1, 2019');  
← undefined  
>> d.toLocaleString()  
← "2019/1/1 上午12:00:00"  
>> d.setDate(9)  
← 1546963200000  
>> d.toLocaleString()  
← "2019/1/9 上午12:00:00"  
>> d.setDate(32)  
← 1548950400000  
>> d.toLocaleString()  
← "2019/2/1 上午12:00:00"  
>> d.setDate(-1)  
← 1548777600000  
>> d.toLocaleString()  
← "2019/1/30 上午12:00:00"
```

凡是涉及到设置月份，都是从0开始算的，即0是1月，11是12月

Set类方法的参数都会自动折算。以setDate为例，如果参数超过当月的最大天数，则向下一个 month 顺延，如果参数是负数，表示从上个月的最后一天开始减去的天数

数组对象Array

- Array是JavaScript的原生对象，同时也是一个构造函数，可以用它生成新的数组
- Array构造函数输入不同的参数，行为不一致

```
>> new Array() // 无参数时，返回一个空数组
```

```
← ▶ Array []
```

```
>> new Array(4) // 单个正整数参数，表示返回的新数组的长度
```

```
← ▶ Array(4) [ <4 empty slots> ]
```

```
>> new Array(4.3) // 非正整数的数值作为参数，会报错
```

```
⚠ RangeError: invalid array length [详细了解]
```

```
>> // 单个非数值（比如字符串、布尔值、对象等）作为参数，则该参数是返回的新数组的成员  
new Array('abc')
```

```
← ▶ Array [ "abc" ]
```

```
>> // 多参数时，所有参数都是返回的新数组的成员  
new Array('a', 'b', 'c')
```

```
← ▶ Array(3) [ "a", "b", "c" ]
```

数组对象Array

- `Array.isArray`方法返回一个布尔值，表示参数是否为数组，它可以弥补`typeof`运算符的不足：
 - `typeof`运算符只能显示数组的类型是`Object`，而`Array.isArray`方法可以识别数组

```
>> var arr = [1, 2, 3];  
← undefined  
>> typeof arr  
← "object"  
>> Array.isArray(arr)  
← true
```

数组对象Array

- push(), pop()方法

- push方法用于在数组的末端添加一个或多个元素，并返回添加新元素后的数组长度
- pop方法用于删除数组的最后一个元素，并返回该元素，对空数组使用pop方法，不会报错，而是返回undefined
- push和pop方法会改变原数组
- push和pop结合使用，就构成了“后进先出”的栈结构 (stack)

数组对象Array

- push(), pop()方法

```
>> var arr = [];  
← undefined  
>> arr.push(1)  
← 1  
>> arr.push('abc')  
← 2  
>> arr.push(true, {})  
← 4  
>> arr  
← ▶ Array(4) [ 1, "abc", true, {} ]  
>> var x = arr.pop()  
← undefined  
>> x  
← ▶ Object {  }  
>> arr  
← ▶ Array(3) [ 1, "abc", true ]
```


数组对象Array

- shift(), unshift()方法
 - shift()方法用于删除数组的第一个元素，并返回该元素
 - push()和shift()结合使用，就构成了“先进先出”的队列结构 (queue)
 - unshift()方法用于在数组的第一个位置添加元素，并返回添加新元素后的数组长度

数组对象Array

- `shift()`, `unshift()`方法

```
>> var arr = [];  
< undefined  
>> arr.push('a','b','c')  
< 3  
>> arr.shift()  
< "a"  
>> arr  
< ► Array [ "b", "c" ]  
>> arr.unshift('aa')  
< 3  
>> arr  
< ► Array(3) [ "aa", "b", "c" ]
```

数组对象Array

- `join()`方法以指定参数作为分隔符，将所有数组成员连接为一个字符串返回
 - 如果不提供参数，默认用逗号分隔
 - 如果数组成员是`undefined`或`null`或空位，会被转成空字符串

```
>> var a = [1, 2, 3, undefined, null, 4];  
← undefined  
>> a.join('|')  
← "1|2|3|||4"  
>> a.join()  
← "1,2,3,,,4"
```

数组对象Array

- **concat**方法用于多个数组的合并。它将新数组的成员，添加到原数组成员的后部，然后**返回一个新数组，原数组不变**
- 除了数组作为参数，**concat**也接受其他类型的值作为参数，添加到目标数组尾部

```
>> ['hello'].concat(['world'], ['!'])  
← ► Array(3) [ "hello", "world", "!" ]  
  
>> [1, 2, 3].concat(4, 5, 6)  
← ► Array(6) [ 1, 2, 3, 4, 5, 6 ]
```

```
> var a = [1,2,3]  
  a.concat(3,4,5)  
-----  
< ► (6) [1, 2, 3, 3, 4, 5]  
-----  
> a  
-----  
< ► (3) [1, 2, 3]
```

数组对象 Array

- **reverse**方法用于颠倒排列数组元素，返回改变后的数组。注意，**该方法将改变原数组**

```
>> var a = ['a', 'b', 'c'];  
← undefined  
>> a.reverse()  
← ▶ Array(3) [ "c", "b", "a" ]  
>> a  
← ▶ Array(3) [ "c", "b", "a" ]
```

数组对象Array

- slice方法用于提取目标数组的一部分，**返回一个新数组**，原数组不变
- slice(start, end)的第一个参数为起始位置（从0开始），第二个参数为终止位置（但该位置的元素本身不包括在内）
 - 如果省略所有参数，返回原数组的拷贝
 - 如果省略第二个参数，则一直返回到原数组的最后一个成员
 - 如果slice方法的参数是负数，则表示倒数的位置
 - 如果第一个参数大于等于数组长度，或者第二个参数小于第一个参数，则返回空数组

数组对象Array

- slice方法

```
>> var a = ['a', 'b', 'c'];  
← undefined  
>> a.slice(1)  
← ▶ Array [ "b", "c" ]  
>> a.slice(1,2)  
← ▶ Array [ "b" ]  
>> a.slice(2,6)  
← ▶ Array [ "c" ]  
>> a.slice()  
← ▶ Array(3) [ "a", "b", "c" ]  
>> a.slice(-2) //表示从倒数第二个位置开始到最后  
← ▶ Array [ "b", "c" ]  
>> a.slice(-2, -1) //表示从倒数第二个位置开始到倒数第一个位置（不包括）  
← ▶ Array [ "b" ]  
>> a.slice(4) //第一个参数大于数组长度，返回空数组  
← ▶ Array []  
>> a.slice(2, 1) //第二个参数小于第一个参数，返回空数组  
← ▶ Array []
```

数组对象Array

- splice方法用于删除原数组的一部分成员，并可以在删除的位置添加新的数组成员，返回值是被删除的元素。**注意，该方法会改变原数组**
 - splice(start, count, addElement1, addElement2, ...)
 - 第一个参数是删除的起始位置（从0开始），第二个参数是被删除的元素个数。如果后面还有更多的参数，则表示这些就是要被插入数组的新元素
 - 起始位置是负数表示从倒数位置开始删除
 - 第二个参数设为0表示单纯地插入元素
 - 只提供第一个参数表示删除到结尾

数组对象Array

- splice方法

```
>> var arr = ['a', 'b', 'c', 'd', 'e', 'f'];
< undefined
>> arr.splice(4,2) //从原数组4号位置（注，从0开始），删除了两个数组成员
< ▶ Array [ "e", "f" ]
>> arr
< ▶ Array(4) [ "a", "b", "c", "d" ]
>> arr.splice(0, 1, 'aa') //从0号位置删除1个元素，并添加'aa'
< ▶ Array [ "a" ]
>> arr
< ▶ Array(4) [ "aa", "b", "c", "d" ]
>> arr.splice(0, 1, 'tt', 'tt') //从0号位置删除1个元素'aa'，并添加'tt','ttt'
< ▶ Array [ "aa" ]
>> arr
< ▶ Array(5) [ "tt", "tt", "b", "c", "d" ]
>> arr.splice(-3,2) //从倒数第3个位置开始，删除2个成员
< ▶ Array [ "b", "c" ]
>> arr
< ▶ Array(3) [ "tt", "tt", "d" ]
>> arr.splice(1,0,"Zzzzz")
< ▶ Array []
>> arr
< ▶ Array(4) [ "tt", "Zzzzz", "tt", "d" ]
>> arr.splice(2)
< ▶ Array [ "tt", "d" ]
>> arr
< ▶ Array [ "tt", "Zzzzz" ]
```

数组对象Array

- **sort**方法对数组成员进行排序，默认是按照字典顺序排序。排序后，**原数组将被改变**
 - 数值会被先转成字符串，再按照字典顺序进行比较

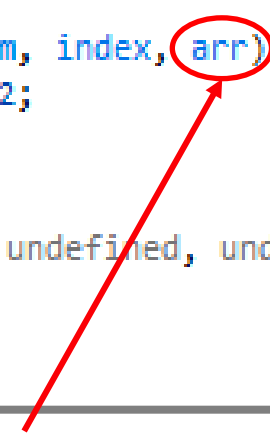
```
>> var arr = ['d', 'c', 'b', 'a'];  
← undefined  
>> arr.sort()  
← ▶ Array(4) [ "a", "b", "c", "d" ]  
>> var arr = [111, 101];  
← undefined  
>> //sort方法不是按照大小排序，而是按照字典顺序  
//数值会被先转成字符串，再按照字典顺序进行比较  
//所以101排在111之前  
arr.sort();  
← ▶ Array [ 101, 111 ]  
>> arr  
← ▶ Array [ 101, 111 ]
```

数组对象Array

- **map**方法将数组的所有成员依次传入参数函数，然后把每一次的执行结果组成一个新数组返回，原数组没有变化
 - 函数调用时，**map**方法向它传入三个参数：当前成员、当前位置和数组本身

```
>> var numbers = [1, 2, 3];  
← undefined  
>> numbers.map(function (n) {  
    return n + 1;  
});  
← ▶ Array(3) [ 2, 3, 4 ]  
>> numbers 原数组没有变化  
← ▶ Array(3) [ 1, 2, 3 ]
```

```
>> var numbers = [1, 2, 3];  
← undefined  
>> numbers.map(function(elem, index, arr) {  
    arr[index] = elem * 2;  
    return;  
});  
← ▶ Array(3) [ undefined, undefined, undefined ]  
>> numbers  
← ▶ Array(3) [ 2, 4, 6 ]
```



数组本身被当成参数传入函数，因此
可以在函数内修改原始数组

数组对象Array

- 如果数组有空位，map方法的回调函数在这个位置不会执行，**会跳过数组的空位，但不会跳过undefined和null**

```
>> var f = function (n) { return 'a' };  
← undefined  
>> [1, undefined, 2].map(f)  
← ▶ Array(3) [ "a", "a", "a" ]  
>> [1, null, 2].map(f)  
← ▶ Array(3) [ "a", "a", "a" ]  
>> [1, , 2].map(f)  
← ▶ Array(3) [ "a", <1 empty slot>, "a" ]
```

数组对象Array

- `forEach`的用法与`map`方法一致，参数是一个函数，该函数同样接受三个参数：当前值、当前位置、整个数组
- `forEach`方法也会跳过数组的空位，但不会跳过`undefined`和`null`

```
>> function log(element, index, array) {  
    console.log('[' + index + '] = ' + element);  
}  
  
[1, 2, , null, undefined, 3].forEach(log);  
[0] = 1  
[1] = 2  
[3] = null  
[4] = undefined  
[5] = 3
```

空位被跳过了

数组对象Array

- **filter**方法用于过滤数组成员，满足条件的成员组成一个新数组返回，该方法**不会改变原数组**
 - 它的参数是一个函数，函数有三个参数：当前成员，当前位置和整个数组
 - 所有数组成员依次执行该函数，**返回结果为true的成员组成一个新数组返回**

```
>> [1, 2, 3, 4, 5].filter(function (elem) {  
    return (elem > 3);  
})  
← ► Array [ 4, 5 ]  
  
>> [1, 2, 3, 4, 5].filter(function (elem, index, arr) {  
    return index % 2 === 0;  
});  
← ► Array(3) [ 1, 3, 5 ]
```

数组对象Array

- **some(), every()**

- 这两个方法类似“断言” (assert)，返回一个布尔值，表示**判断数组成员是否符合某种条件**
- 接受一个函数作为参数，所有数组成员依次执行该函数，该函数有三个参数：当前成员、当前位置和整个数组，然后返回一个布尔值
- **some**方法判断**是否存在符合条件的成员**，只要一个成员的返回值是true，则整个**some**方法的返回值就是true，否则返回false
- **every**方法判断**是否所有成员都符合条件**，所有成员的返回值都是true，整个**every**方法才返回true，否则返回false

数组对象Array

- `some()`, `every()`

```
>> var arr = [1, 2, 3, 4, 5];  
    arr.some(function (elem, index, arr) {  
      return elem >= 3;  
    });  
← true  
  
>> var arr = [1, 2, 3, 4, 5];  
    arr.every(function (elem, index, arr) {  
      return elem >= 3;  
    });  
← false
```

```
> var arr = [1, 2, 3, 4, 5];  
  arr.some(function (n)  
    {return n >= 3;});  
  
← true
```

逻辑只用到了 **elem**，根本不需要 **index** 和 **arr**。
因此可以把多余参数去掉，写成更短的

数组对象Array

- **reduce(), reduceRight()**
 - reduce方法和reduceRight方法依次处理数组的每个成员，最终**累计为一个值**
 - reduce是从左到右处理（从第一个成员到最后一个成员），reduceRight则是从右到左（从最后一个成员到第一个成员）

```
>> var xx = [1, 2, 3, 4, 5].reduce(function (a, b) {  
    console.log(a, b);  
    return a + b;  
})  
1 2  
3 3  
6 4  
10 5  
← undefined  
>> xx  
← 15
```

return 的值会作为下一个循环的累积值a传进去。

```
>> var xx = [1, 2, 3, 4, 5].reduceRight(function (a, b) {  
    console.log(a, b);  
    return a - b;  
})  
5 4  
1 3  
-2 2  
-4 1  
← undefined  
>> xx  
← -5
```

数组对象Array

- **indexOf(), lastIndexOf()**
 - indexOf方法返回给定元素在数组中第一次出现的位置，如果没有出现则返回-1
 - lastIndexOf方法返回给定元素在数组中最后一次出现的位置，如果没有出现则返回-1
 - 方法都可以接受第二个参数，表示搜索的开始位置
 - 两个方法内部使用严格相等运算符进行比较

反向搜索

```
>> ['a', 'b', 'c'].indexOf('a')  
< 0  
>> ['a', 'b', 'c'].indexOf('a', 1)  
< -1  
>> ['a', 'b', 'c'].lastIndexOf('a')  
< 0  
>> ['a', 'b', 'c'].lastIndexOf('a', 2)  
< 0
```

Math 对象

- Math是 JavaScript 的原生对象，提供各种数学功能
- 该对象不是构造函数，不能生成实例，所有的属性和方法都必须在Math对象上调用

Math 对象

- Math对象的静态属性，提供以下数学常数，这些属性都是只读的，不能修改

- `Math.E` : 常数 e 。
- `Math.LN2` : 2 的自然对数。
- `Math.LN10` : 10 的自然对数。
- `Math.LOG2E` : 以 2 为底的 e 的对数。
- `Math.LOG10E` : 以 10 为底的 e 的对数。
- `Math.PI` : 常数 π 。
- `Math.SQRT1_2` : 0.5 的平方根。
- `Math.SQRT2` : 2 的平方根。

```
Math.E // 2.718281828459045
Math.LN2 // 0.6931471805599453
Math.LN10 // 2.302585092994046
Math.LOG2E // 1.4426950408889634
Math.LOG10E // 0.4342944819032518
Math.PI // 3.141592653589793
Math.SQRT1_2 // 0.7071067811865476
Math.SQRT2 // 1.4142135623730951
```

Math 对象

- **Math对象的静态方法：**

- `Math.abs()` : 绝对值
- `Math.ceil()` : 向上取整
- `Math.floor()` : 向下取整
- `Math.max()` : 最大值
- `Math.min()` : 最小值
- `Math.pow()` : 指数运算
- `Math.sqrt()` : 平方根
- `Math.log()` : 自然对数
- `Math.exp()` : `e` 的指数
- `Math.round()` : 四舍五入
- `Math.random()` : 随机数

Math 对象

- Math对象的静态方法:

```
>> Math.abs(-1) //返回参数值的绝对值
< 1
>> Math.max(2, -1, 5)
< 5
>> Math.min(2, -1, 5)
< -1
>> Math.floor(3.2) //返回小于参数值的最大整数
< 3
>> Math.floor(-3.2) //返回小于参数值的最大整数
< -4
>> Math.ceil(3.2) //返回大于参数值的最小整数
< 4
>> Math.ceil(-3.2) //返回大于参数值的最小整数
< -3
>> //一个总是返回数值的整数部分的函数
function ToInteger(x) {
  x = Number(x);
  return x < 0 ? Math.ceil(x) : Math.floor(x);
}
< undefined
>> ToInteger(3.5)
< 3
>> ToInteger(-3.2)
< -3
```

```
>> Math.round(0.5) //四舍五入
< 1
>> Math.round(-0.5) //四舍五入
< -0
>> Math.round(-0.8) //四舍五入
< -1
```

注意对0.5的四舍五入
是如何处理的

```
>> Math.pow(2, 3) //2的3次幂
< 8
>> Math.sqrt(4) //返回参数值的平方根
< 2
>> Math.sqrt(-4)
< NaN
>> Math.log(Math.E) //返回以e为底的自然对数值
< 1
>> Math.log(100)/Math.LN10 //要计算以10为底的对数，可以先用Math.log求出自然对数，然后除以Math.LN10
< 2
>> Math.log(8)/Math.LN2 //求以2为底的对数，可以除以Math.LN2
< 3
>> Math.exp(1) //返回常数e的参数次方
< 2.7182818284590455
```

Math 对象

- `Math.random()` 返回 **[0,1)** 之间的一个伪随机数

```
Math.random() // 0.7151307314634323
```

可能等于0，但是一定小于1

- 任意范围的随机数生成函数如下：

```
function getRandomArbitrary(min, max) {  
  return Math.random() * (max - min) + min;  
}
```

```
getRandomArbitrary(1.5, 6.5)  
// 2.4942810038223864
```

- 任意范围的随机整数生成函数如下

```
function getRandomInt(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

```
getRandomInt(1, 6) // 5
```

Math 对象

- Math对象还提供一系列三角函数方法：

- `Math.sin()` : 返回参数的正弦 (参数为弧度值)
- `Math.cos()` : 返回参数的余弦 (参数为弧度值)
- `Math.tan()` : 返回参数的正切 (参数为弧度值)
- `Math.asin()` : 返回参数的反正弦 (返回值为弧度值)
- `Math.acos()` : 返回参数的反余弦 (返回值为弧度值)
- `Math.atan()` : 返回参数的反正切 (返回值为弧度值)

```
Math.sin(0) // 0
Math.cos(0) // 1
Math.tan(0) // 0

Math.sin(Math.PI / 2) // 1

Math.asin(1) // 1.5707963267948966
Math.acos(1) // 0
Math.atan(1) // 0.7853981633974483
```


JavaScript与DOM

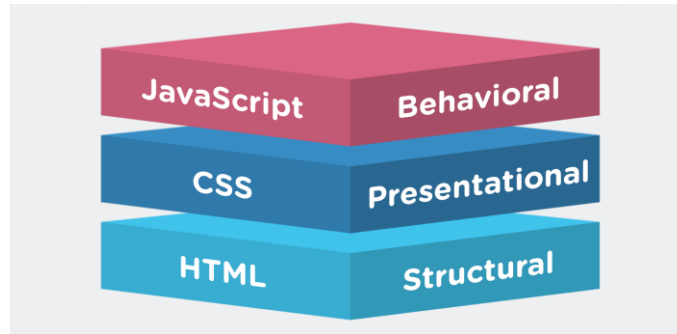


主要内容

- 文档对象模型DOM
- 事件
- 浏览器对象模型BOM

DOM在Web标准中的地位

- 一个Web页面或者Web应用程序由三部分组成：HTML、CSS和JavaScript。如果把一个Web页比作一套房子，那么房子的格局就是HTML，房子装修就是CSS，房门怎么开、灯怎么操作就是JavaScript。其中的JavaScript赋予了Web页面的行为，通过它可以与页面进行交互，实现页面动画效果等



- 要实现页面的动态交互和效果，操作HTML是核心，操作HTML其实就是对DOM的操作。**DOM提供了用程序来动态控制HTML的接口**

什么是 DOM?

- DOM (Document Object Model, 文档对象模型) 是 W3C (万维网联盟) 的标准
- DOM 是一个用于 HTML 和 XML 文档的应用程序编程接口(API), 它定义了文档的逻辑结构以及访问和操作文档(改变文档的结构、样式和内容)的方式
- 在规范中, 术语 “文档” 更多的是广义上的文档
 - XML 常被用作表示许多不同类型的信息的一种方式, 这些信息可能存储在不同的系统中, 其中的大部分被看作是数据而不是文档
 - 尽管如此, XML 可以将这些数据作为文档呈现, 并且可以使用 DOM 来管理这些数据

DOM提供的能力

- DOM允许程序和脚本动态地访问和更新文档的内容、结构和样式
- 使用文档对象模型（DOM），程序员可以构建文档、导航其结构以及添加、修改或删除元素和内容
- 除了少数例外，在HTML或XML文档中找到的任何东西都可以使用文档对象模型访问、更改、删除或添加

DOM是中立于平台和语言的接口

- 作为一个W3C规范，DOM的一个重要目标是提供一个可用于各种环境和应用程序的标准编程接口，**DOM**被设计成与特定编程语言相独立，DOM接口规范可以用各种语言实现

```
1 paragraphs = document.getElementsByTagName("P");  
2 // paragraphs[0] is the first <p> element  
3 // paragraphs[1] is the second <p> element, etc.  
4 alert(paragraphs[0].nodeName);
```

JavaScript实现

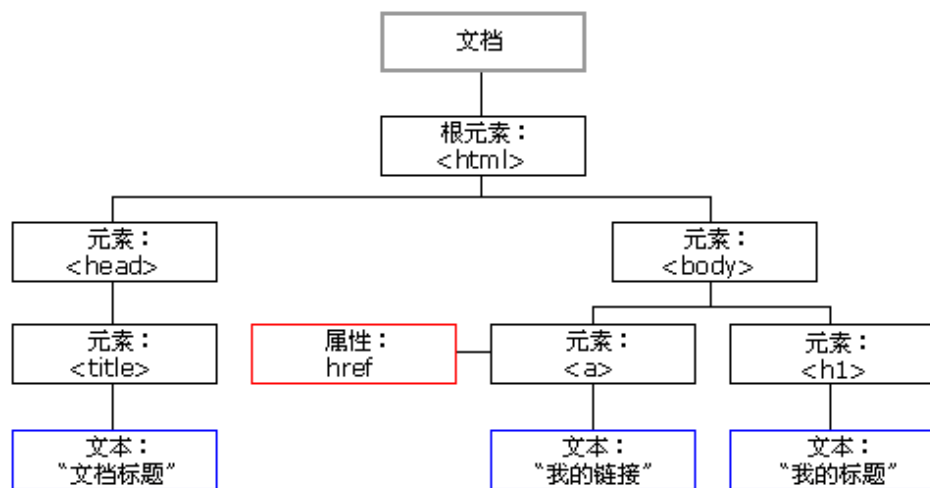
JavaScript 是最常用于 DOM 操作的语言，也是本课程学习的主要内容

```
1 # Python DOM example  
2 import xml.dom.minidom as m  
3 doc = m.parse("C:\\Projects\\Py\\chap1.xml");  
4 doc.nodeName # DOM property of document object;  
5 p_list = doc.getElementsByTagName("para");
```

Python实现

DOM 示例

- 在 HTML DOM 中，所有事物都是节点，浏览器会根据 DOM 模型，将结构化文档解析成一系列的节点，再由这些节点组成一个树状结构（DOM Tree），所有的节点和最终的树状结构，都有规范的对外接口



- 整个文档是一个文档节点
- 每个 HTML 元素是元素节点
- HTML 元素内的文本是文本节点
- 每个 HTML 属性是属性节点
- 注释是注释节点

DOM 节点

- DOM 的最小组成单位叫做节点 (node)，文档的树形结构 (DOM 树) 由各种不同类型的节点组成，每个节点是文档树的叶子
- DOM 节点的主要类型有七种：
 - Document : 整个文档树的顶层节点
 - DocumentType : doctype 标签 (比如 `<!DOCTYPE html>`)
 - Element : 网页的各种HTML标签 (比如 `<body>`、`<a>` 等)
 - Attribute : 网页元素的属性 (比如 `class="right"`)
 - Text : 标签之间或标签包含的文本
 - Comment : 注释
 - DocumentFragment : 文档的片段

DOM 节点

- 元素节点：Element

- 元素节点是组成文档树的重要部分，它表示了HTML、XML文档中的元素。通常元素有子元素、文本节点或者两者的结合，**元素节点是唯一能够拥有属性的节点类型**。html、head、title、body、div、ul、li等都属于Element（元素节点）

- 属性节点：Attr

- 属性节点代表了元素中的属性，因为属性实际上是附属于元素的，属性节点其实被看做是包含它的元素节点的一部分，它并不作为单独的一个节点在文档中出现。**HTML元素的属性如id、class、href等都属于Attr（属性节点）**

- 文本节点：Text

- 文本节点是只包含文本内容的节点，**文本节点可以只包含空格**。在文档树中元素的文本内容和属性的文本内容都是由文本节点来表示

- 注释节点：Comment

- 注解节点其实就是我们平时所说的注释内容

DOM 节点

- 文档节点：Document

- 文档节点是文档树的根节点，它是文档中其他所有节点的父节点。文档节点并不是HTML、XML文档的根元素，因为在XML文档中，处理指令、注释等内容可以出现在根元素之外，所以在构造DOM树的时候，根元素并不适合作为根节点，因此就有了文档节点，而根元素是作为文档节点的子节点出现的。比如例子中的<!DOCTYPE html>、html作为Document（文档节点）的子节点出现

- 文档类型节点：DocumentType

- 每个Document都有一个DocumentType属性，它的值或是null，或者是DocumentType对象。比如声明文档类型时<!DOCTYPE html>就是文档类型节点。

- 文档片段节点：DocumentFragment

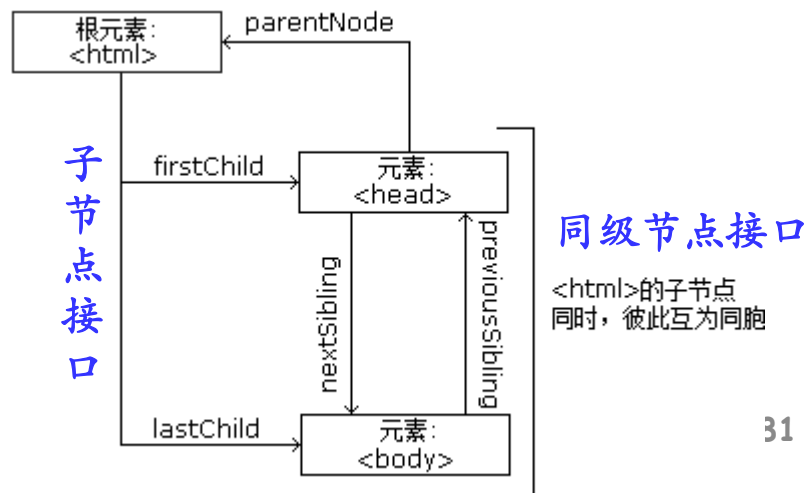
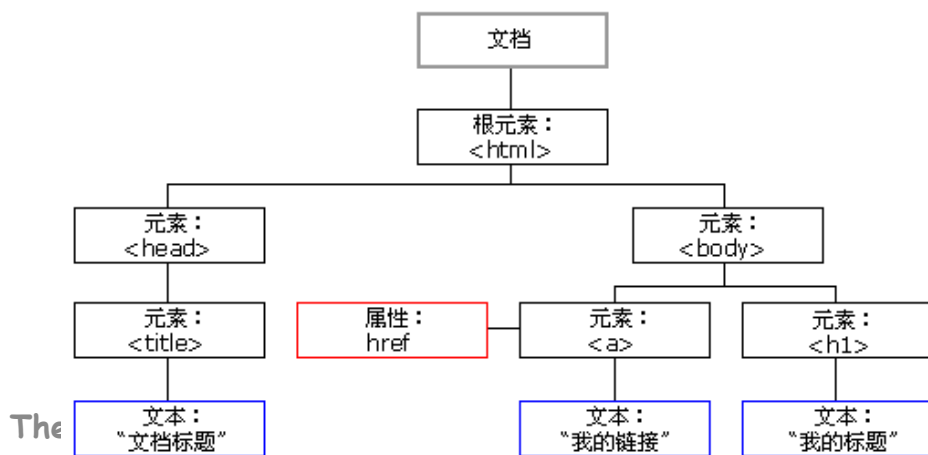
- 文档片段节点是轻量级的或最小的Document对象，它表示文档的一部分或者是一段，不属于文档树。它的特殊性在于，当请求把一个DocumentFragment节点插入到文档的时候，插入的不是DocumentFragment自身，而是它的所有子孙节点。这使得DocumentFragment可以用来暂时存放那些一次插入文档的节点，有利于实现文档的剪切、复制和粘贴等操作

- <!DOCTYPE> 声明不是 HTML 标签，它是指示 web 浏览器关于页面使用哪个 HTML 版本进行编写的指令
- <!DOCTYPE> 声明必须是 HTML 文档的第一行，位于 <html> 标签之前，没有结束标签，对大小写不敏感
- <!DOCTYPE html> 是最短的有效文档声明

```
XHTML <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
      "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
HTML5 <!DOCTYPE html>
```

DOM树

- 浏览器原生提供document节点代表整个文档
- DOM树的第一层只有一个节点，它构成了树结构的根节点（root node），其他HTML标签节点都是它的下级节点
- 除了根节点和根元素，其他节点都有三种层级关系，DOM提供操作接口，用来获取这三种关系的节点
 - 父节点关系（parentNode）：直接的那个上级节点
 - 子节点关系（childNodes）：直接的下级节点
 - 兄弟节点关系（sibling）：拥有同一个父节点的节点



DOM主要内容

- DOM节点相关接口
 - DOM节点Node：主要属性、操作节点的常用操作
 - DOM节点集合NodeList和HTMLCollection
 - ParentNode和ChildNode接口
- DOM中的主要对象
 - Document节点对象
 - 元素节点（Element）对象及属性、样式操作
 - Text节点和 DocumentFragment 节点

Node 接口

- 所有 DOM 节点对象都继承了 Node 接口，拥有一些共同的属性和方法，这是 DOM 操作的基础

属性

- nodeType
- nodeName
- nodeValue
- textContent
- baseURI
- ownerDocument
- nextSibling
- previousSibling
- parentNode
- parentElement
- firstChild, lastChild
- childNodes
- isConnected

方法

- appendChild()
- hasChildNodes()
- cloneNode()
- insertBefore()
- removeChild()
- replaceChild()
- contains()
- compareDocumentPosition()
- isEqualNode(), isSameNode()
- normalize()
- getRootNode()

nodeType属性

- **nodeType**属性返回一个整数值，表示节点类型，**Node**对象定义了几个常量，对应这些类型值
 - 文档节点 (document) : 9 , 对应常量 `Node.DOCUMENT_NODE`
 - 元素节点 (element) : 1 , 对应常量 `Node.ELEMENT_NODE`
 - 属性节点 (attr) : 2 , 对应常量 `Node.ATTRIBUTE_NODE`
 - 文本节点 (text) : 3 , 对应常量 `Node.TEXT_NODE`
 - 文档片断节点 (DocumentFragment) : 11 , 对应常量 `Node.DOCUMENT_FRAGMENT_NODE`
 - 文档类型节点 (DocumentType) : 10 , 对应常量 `Node.DOCUMENT_TYPE_NODE`
 - 注释节点 (Comment) : 8 , 对应常量 `Node.COMMENT_NODE`
- 使用**nodeType**属性确定节点类型是常用方法

```
var node = document.documentElement.firstChild;
if (node.nodeType === Node.ELEMENT_NODE) {
    console.log('该节点是元素节点');
}
```

```
> var node = document.documentElement.firstChild;
if (node.nodeType === 1) {
    console.log('该节点是元素节点');}
该节点是元素节点
```

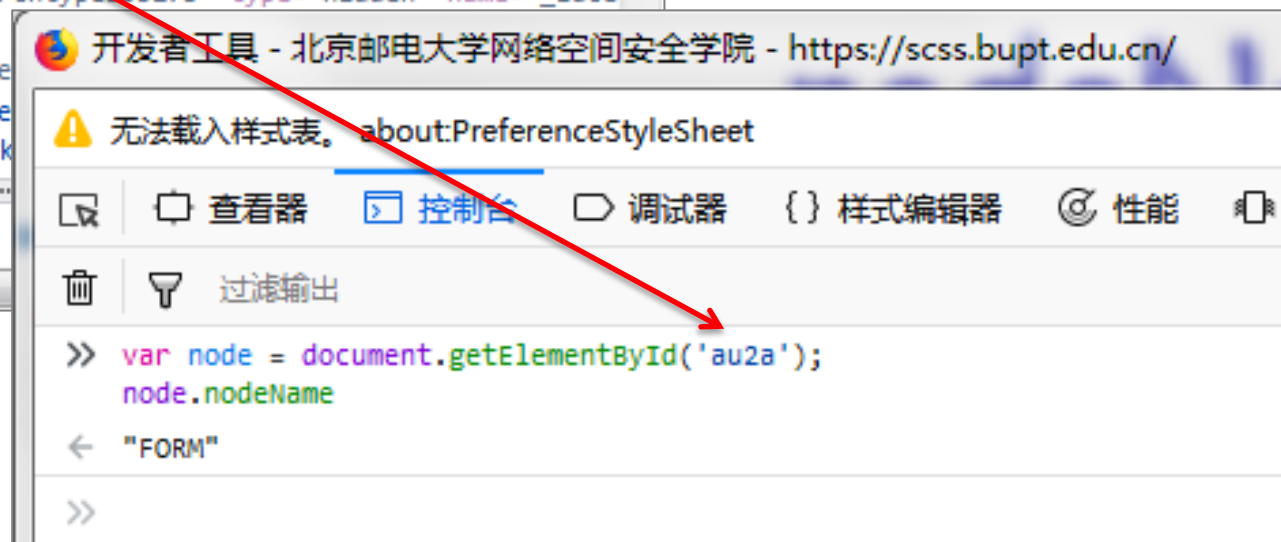
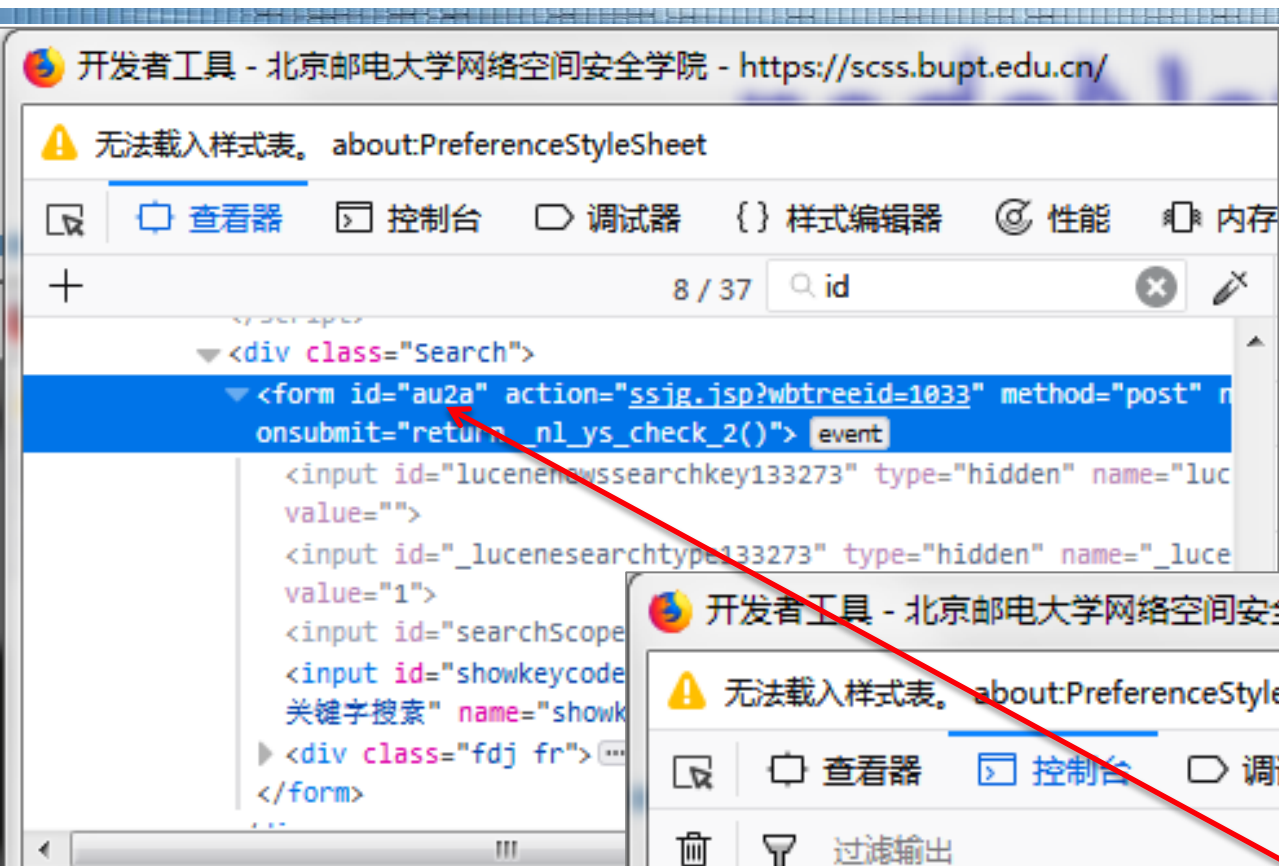
nodeName 属性

- nodeName 属性返回节点的名称，不同节点的 nodeName 属性值如下：

- 文档节点 (document) : #document
- 元素节点 (element) : 大写的标签名
- 属性节点 (attr) : 属性的名称
- 文本节点 (text) : #text
- 文档片断节点 (DocumentFragment) : #document-fragment
- 文档类型节点 (DocumentType) : 文档的类型
- 注释节点 (Comment) : #comment

```
>> document.nodeName  
← "#document"
```


nodeName 属性



nodeValue属性

- nodeValue属性返回一个字符串，表示当前节点本身的文本值，该属性可读写
- 只有文本节点（text）、注释节点（comment）和属性节点（attr）有文本值，因此这三类节点的nodeValue可以返回结果，其他类型的节点一律返回null
- 同样也只有这三类节点可以设置nodeValue属性的值，其他类型的节点设置无效

textContent 属性

- textContent 属性返回 **当前节点和它的所有后代节点** 的文本内容
 - 对于文本节点 (text)、注释节点 (comment) 和属性节点 (attr)，textContent 属性的值与 nodeValue 属性相同
 - 对于其他类型的节点，该属性会将每个子节点（不包括注释节点）的内容连接在一起返回。如果一个节点没有子节点，则返回空字符串
- 该属性是 **可读写的**，设置该属性的值，会用一个新的文本节点，替换所有原来的子节点。

```
16 <div id="d0">hello world</div>  
17 <div id="divA">This is <span>some</span> text</div>  
18
```

hello world
This is some text

开发者工具 - DOM test page - file:///C:/我的工作目录/北邮工

查看器 控制台 调试器 {} 样式编辑器

过滤输出

```
>> document.getElementById('divA').textContent
```

```
< "This is some text"
```

```
>>
```

子节点内容连接

baseURI 属性

- **baseURI** 属性返回一个字符串，表示 **当前网页的绝对路径**。浏览器根据这个属性，计算网页上的相对路径的 **URL**。该属性为 **只读**

```
>> document.baseURI
< "file:///C:/%E6%88%91%E7%9A%84%E5%B7%A5%E4%BD%9C%E7%9B%AE%E5%BD%95/%E5%8C%97%E9%82%AE%E5%B7%A5%E4%BD%9C/Web%E6%8A%80%E6%9C%AF%E5%BC%80%E5%8F%91%E5%9F%BA%E7%A1%80%20-%20%E8%AF%BE%E7%A8%8B/%E8%AF%BE%E7%A8%8BPPT/src/dom/domtest.html"
>> var sbaseURI=document.baseURI
< undefined
>> decodeURI(sbaseURI)
< "file:///C:/我的工作目录/北邮工作/Web技术开发基础 - 课程/课程PPT/src/dom/domtest.html"
```

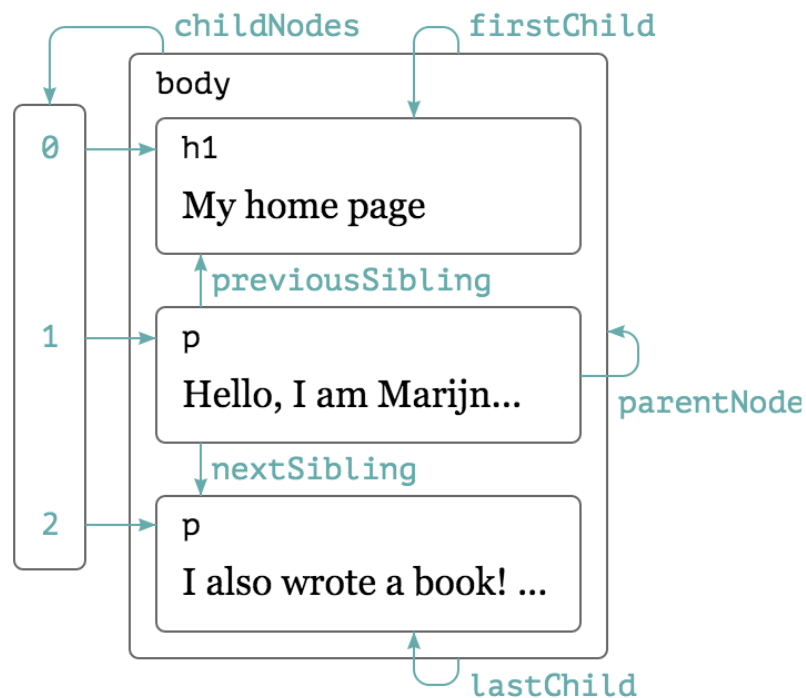
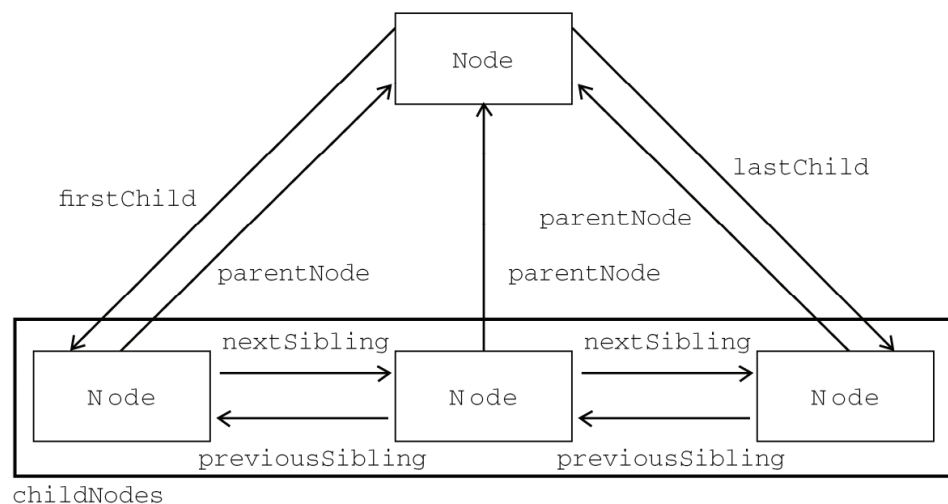
ownerDocument 属性

- Node.ownerDocument 属性返回当前节点所在的顶层文档对象，即 document 对象

```
>> var div = document.getElementById('d0');  
← undefined  
>> div.ownerDocument  
← ▶ HTMLDocument file:///C:/E6%88%91%E7%9A%84%E5%B7%A5%E4%BD%9C%E7%9B%AE%E5%BD%95/%E5%8C%97%E9%82%AE%E5%B7%A5%E4%BD%9C/Web%E6%8A%80%E6%9C%AF%E5%BC%80%E5%8F%91%E5%9F%BA%E7%A1%80%20-%20%E8%AF%BE%E7%A8%8B/%E8%AF%BE%E7%A8%8BPPT/src/dom/domtest.html
```

利用关系属性遍历DOM树

- 可以通过使用parentNode、firstChild、lastChild、previousSibling和nextSibling等属性来实现遍历DOM树



nextSibling属性

- nextSibling属性返回紧跟在当前节点后面的第一个同级节点，如果当前节点后面没有同级节点，则返回null

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
5   <title>DOM example</title>
6 </head>
7
8 <body>
9   <div id="d0">hello world</div>
10  <div id="divA">This is <span>some</span> text</div>
11  <div id="d1">hello</div><div id="d2">world</div>
12 </body>
13 </html>
```

开发者工具 - DOM example - file:///C:/我的工作目录/北邮工作/Web技术开发基础 - 课程/课程PPT/src/dom/

查看器 控制台 调试器 {} 样式编辑器 性能 内存 网络 存储

过滤输出

```
>> var d1 = document.getElementById('d1');
    var d2 = document.getElementById('d2');
< undefined
>> d1.nextSibling === d2
< true
>>
```

nextSibling属性

- 该属性还包括文本节点和注释节点，如果当前节点后面有空格，该属性会返回一个文本节点，内容为空格

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
5   <title>DOM example</title>
6 </head>
7
8 <body>
9   <div id="d0">hello world</div>
10  <div id="divA">This is <span>some</span> text</div>
11  <div id="d1">hello</div> <div id="d2">world</div>
12 </body>
13 </html>
```

开发者工具 - DOM example - file:///C:/我的工作目录/北部工作/Web技术开发基础 - 课程/课程PPT/src/dom/

查看器 控制台 调试器 {} 样式编辑器 性能 内存 网络 存储

过滤输出

```
>> var d1 = document.getElementById('d1');
    var d2 = document.getElementById('d2');
< undefined
>> d1.nextSibling
< #text " "
>> d1.nextSibling.nodeValue
< " "
```

对比上页的示例，多了两个空格，
则d1的nextSibling节点就不再是d2，
而是文本节点

nextSibling属性

- nextSibling属性可以用来遍历所有子节点

```
9      <div id="d0">hello world</div>
10     <div id="divA">This is <span>some</span> text</div>
11     <div id="d1">hello</div> <div id="d2">world</div>
12 </body>
13 </html>
```

开发者工具 - DOM example - file:///C:/我的工作目录/北邮工作/Web技术开发基础 -

查看器 控制台 调试器 样式编辑器 性能 内存

过滤输出

```
>> var el = document.getElementById('divA').firstChild;

while (el !== null) {
  console.log(el.nodeName);
  el = el.nextSibling;
}

#text
SPAN
#text
← null
```

firstChild 是文本节点 "This is "，输出 #text。
nextSibling 是 元素，输出 SPAN。
nextSibling 再次指向文本节点 "text"，输出 #text。
最后，nextSibling 变为 null，循环结束。

previousSibling 属性

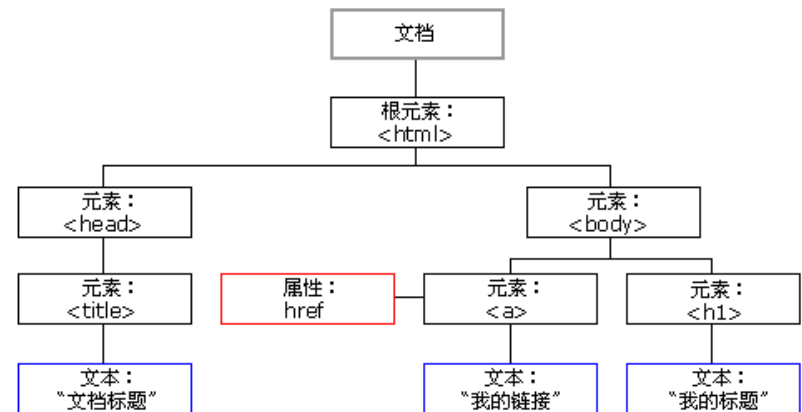
- previousSibling 属性返回当前节点**前面的、距离最近的一个同级节点**。如果当前节点前面没有同级节点，则返回null
- 和nextSibling属性一样，该属性还**包括文本节点和注释节点**。因此如果当前节点前面有空格，该属性会返回一个文本节点，内容为空格

parentNode和parentElement属性

- parentNode属性返回当前节点的父节点，对于一个节点来说，它的父节点只可能是三种类型：
 - 元素节点 (element)
 - 文档节点 (document)
 - 文档片段节点 (documentfragment)
 - DocumentFragment节点代表文档片段，是一个完整的DOM 树形结构，它不属于当前文档，没有父节点，但可以插入任意数量的子节点，一般用于构建一个 DOM 结构，然后插入当前文档
- parentElement属性返回当前节点的父元素节点，如果当前节点没有父节点，或者父节点类型不是元素节点，则返回null

parentNode和parentElement属性

```
1  <!DOCTYPE html>
2  <html>
3  ⊕ <head> ...
6  </head>
7
8  <body>
9      <div id="d0">hello world</div>
10     <div id="divA">This is <span>some</span> text</div>
11     <div id="d1">hello</div> <div id="d2">world</div>
12 </body>
13 </html>
```



```
>> var d0 = document.getElementById('d0')
```

```
← undefined
```

```
>> d0.parentNode
```

```
← ▶ <body> 📦
```

```
>> d0.parentNode.parentNode
```

```
← ▶ <html> 📦
```

```
>> d0.parentNode.parentElement
```

```
← ▶ <html> 📦
```

```
>> d0.parentNode.parentNode.parentNode
```

```
← ▶ HTMLDocument file:///C:/E6%88%91%E7%9A%84%E5%B7%A5%E4%BD%9C%E7%9B%AE%E5%BD%95/%E5%8C%97%E9%82%AE%E5%B7%A5%E4%BD%9C/Web%E6%8A%80%E6%9C%AF%E5%BC%80%E5%8F%91%E5%9F%BA%E7%A1%80%20-%20%E8%AF%BE%E7%A8%8B/%E8%AF%BE%E7%A8%8BPPT/src/dom/domtest.html
```

```
>> d0.parentNode.parentNode.parentElement
```

```
← null
```

do.parentNode.parentNode是<html>节点，<html>的父节点是文档节点，不是元素节点，因此do.parentNode.parentNode.parentElement返回null

firstChild和lastChild属性

- firstChild属性返回当前节点的第一个子节点，返回的可能是元素节点、文本节点或注释节点，如果当前节点没有子节点，则返回null
- lastChild属性返回当前节点的最后一个子节点，用法与firstChild属性相同

```
<div id="divA">This is <span>some</span> text</div>
```

```
>> var divA = document.getElementById('divA')
```

```
← undefined
```

```
>> divA.firstChild.nodeValue
```

```
← "This is "
```

```
>> divA.lastChild.nodeValue
```

```
← " text"
```

childNodes 属性

- childNodes 属性返回一个类似数组的对象集合 NodeList，成员包括当前节点的所有子节点

```
>> <<div id="divA">This is <span>some</span> text</div>
var div = document.getElementById('divA');
var children = div.childNodes;

for (var i = 0; i < children.length; i++) {
    console.log(children[i].nodeValue);
}

This is
null
text
```

注释：childNodes 属性的返回值包括元素节点、文本节点和注释节点，**而只有文本节点（text）、注释节点（comment）和属性节点（attr）有文本值**，因此这三类节点的nodeValue可以返回结果，其他类型的节点一律返回null

isConnected属性

- isConnected属性返回一个布尔值，表示当前节点是否在文档之中

```
>> var test = document.createElement('p');  
← undefined  
>> test.isConnected  
← false  
>> document.body.appendChild(test)  
← ▶ <p> □  
>> test.isConnected  
← true
```

test节点是脚本生成的节点，没有插入文档之前，isConnected属性返回false，插入之后返回true

俄罗斯方向网络作战力量介绍

俄罗斯下属情报机构

- FSO--联邦警卫局
- FSB--联邦安全局
- SVR--对外情报局
- GRU--武装力量总参谋部情报总局

APT组织

- APT28--GRU 26165 部队或第85 特殊服务中心
- APT29--FSB 背景
- Turla--疑似具有FSB背景
- Gamaredon—FSB 克里米亚分部
- Dragonfly—FSB 第16 中心 或者 71330 部队
- Sandworm--GRU 74455 部队或特殊技术中心

